Losowa segmentacja obrazów i augmentacja lokalna jako strategia poprawy uczenia głębokiego

Projekt Interdyscyplinarny

Natalia Choszczyk | Mateusz Deptuch | Karolina Dunal | Zofia Kamińska

Politechnika Warszawska

Wydział Matematyki i Nauk Informacyjnych

10 kwietnia 2025

Spis treści

1. Wstęp **2**
1.1. Wprowadzenie i cel projektu 2
1.2. Przegląd literatury 2
2. Pierwsze wersje implementacji augmentorów **4**
2.1. Klasa STESAugmentor 4
2.2. Klasa Augmentor 5
2.2.1. Siatka podziału 5
2.2.2. Tryby augmentacji 6
2.2.3. Augmentacje 6
2.3. Wizualizacja działania augmentorów 6
3. Pierwsze eksperymenty **8**
3.1. Konfiguracje i struktury eksperymentów 8
4. Eksperymenty - ciąg dalszy **11**
4.1. Konfiguracje i struktury kolejnych eksperymentów 11
4.2. Wyniki drugiej serii eksperymentów 11
4.3. Poprawa błędów technicznych w augmentacji obrazów 12
4.4. Kontynuacja eksperymentów 13
4.4.1. Modyfikacja parametru p 18
4.5. Trzecia faza eksperymentów — augmentacje złożone 21
4.5.1. Wyniki trzeciej serii eksperymentów 22
5. Podsumowanie **27**

1. Wstęp

1.1. Wprowadzenie i cel projektu

Inspiracją dla niniejszego projektu było zbadanie, czy połączenie dwóch szeroko stosowanych podejść w uczeniu maszynowym — *augmentacji danych* oraz *curriculum learning* — może potencjalnie poprawić wyniki modeli wizji komputerowej. Planowano zaproponować i przetestować nową metodę augmentacji, opartą na niestandardowym podziale fragmentów obrazu, która w kolejnych etapach projektu miała zostać połączona z konceptem *curriculum learning*. Jednak ze względu na obszerność i złożoność części związanej z implementacją oraz testami samej augmentacji, zasoby czasowe nie pozwoliły na rozwinięcie tego aspektu. W przeglądzie literatury nie znaleźliśmy wielu metod skupiających się na augmentacji wybranych, niesymetrycznych fragmentów obrazu z użyciem różnych technik. Postanowiliśmy zbadać takie podejście, zakładając, że może ono skuteczniej uwzględniać lokalne cechy istotne dla danego zadania.

1.2. Przegląd literatury

W tej sekcji prezentujemy krótkie opisy prac badawczych najbardziej zbliżonych do naszego wyjściowego pomysłu, które posłużyły jako inspiracja oraz punkt odniesienia dla naszego projektu.

- **Model-Adaptive Data Augmentation (MADAug)** [3] wprowadza dynamiczne podejście do augmentacji danych, dostosowując transformacje do aktualnego stanu modelu podczas treningu. Zamiast stosować stałą politykę augmentacyjną, *MADAug* wykorzystuje sieć polityki augmentacyjnej, która wybiera odpowiednie operacje dla każdego obrazu w zależności od etapu treningu. Proces ten jest realizowany poprzez dwupoziomową optymalizację, mającą na celu minimalizację błędu na zbiorze walidacyjnym. Dzięki temu model uczy się efektywniej, zaczynając od prostszych przykładów i stopniowo przechodząc do bardziej złożonych (koncept *curriculum learning*).

- **Colorful Cutout** [2] proponuje technikę augmentacji danych, która zamiast klasycznego wycinania fragmentów obrazu stosuje kolorowe zakłócenia o rosnącej złożoności i intensywności wraz z postępem treningu modelu. Podejście to integruje augmentację danych z koncepcją uczenia według stopnia trudności (*curriculum learning*), umożliwiając modelowi stopniowe adaptowanie się do coraz bardziej wymagających przykładów.

- Praca *Enhancing deep learning classification performance of tongue lesions in imbalanced data: mosaic-based soft labeling with curriculum learning* [5] prezentuje metodę augmentacji danych opartą na mozaikowym wycinaniu fragmentów obrazu oraz miękkim etykietowaniu, połączoną z techniką *curriculum learning*. Metoda ta została zaprojektowana w celu poprawy efektywności klasyfikacji zmian językowych w obrazach medycznych, zwłaszcza w warunkach niezrównoważonych danych.

- **Self-transformation of Encoded Subregion (STES)** [4] to metoda augmentacji danych, opierająca się na przekształcaniu wybranych zakodowanych fragmentów obrazu. Najpierw nakładany jest regularny grid dzielący obraz na małe fragmenty. Następnie losowo wybierany jest jeden fragment spośród czterech sąsiadujących fragmentów tworzących większy kwadrat, który poddawany jest augmentacji. W każdym takim większym kwadracie stosowane są te same operacje transformacji — rotacja oraz zmiana koloru.

Najbliższym podejściem względem naszej metody okazała się metoda **STES**, który również opiera się na przekształceniach fragmentów obrazu, jednak nasze podejście różni się sposobem dzielenia obrazu — nie jest to podział na równe części ani w równych odstępach. Szczegóły działania zaproponowanego przez nas rozwiązania zostaną opisane w kolejnej sekcji. W ramach prac badawczych finalnie skupiliśmy się na analizie wpływu różnych wariantów augmentacji, nie poszerzając badań o zastosowanie koncepcji *curriculum learning*.

2. Pierwsze wersje implementacji augmentorów

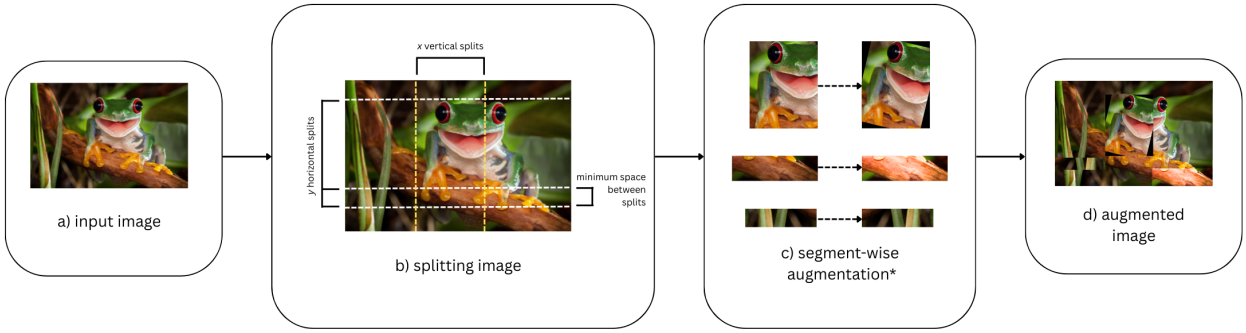
2.1. Klasa STESAugmentor

Zdecydowaliśmy się zaimplementować metodę opisaną w artykule **Self-transformation of Encoded Subregion (STES)** [3], ponieważ była ona najbliższa naszej propozycji i stanowi dobrą bazę porównawczą. Ze względu na brak dostępnego kodu źródłowego autorów, odwzorowanie metody może nie być w pełni dokładne. Mimo to, po wstępnych testach na kilku obrazach, uzyskane rezultaty wizualnie są bardzo zbliżone do tych przedstawionych w pracy.
Schemat działania augmentora można opisać następująco:

1. Wczytaj obraz wejściowy.
2. Podziel obraz na siatkę 2×2 segmentów o losowo wybranym rozmiarze w zakresie $[\alpha L, \beta L]$, gdzie $L = \min(\text{wysokość}, \text{szerokość})$.
3. W każdej czterosegmentowej grupie losowo wybierz jeden segment.
4. Na wybranym segmencie wykonaj:
   - losową transformację przestrzeni barw (zmiana odcienia),
   - rotację o losowy kąt z zakresu $[45°, 315°]$,
   - dopasowanie jasności/kontrastu.
5. Zamień oryginalny segment w obrazie przekształconą wersją.
6. Zapisz i wyświetl obraz wynikowy.

2.2. Klasa Augmentor

W zaproponowanej przez nas metodzie augmentacji założono niestandardowy sposób augmentacji obrazów, w którym główną ideą jest podział obrazu wejściowego na prostokątne regiony i niezależne przekształcanie tych fragmentów. Liczono, że dzięki temu zabiegowi możliwe będzie uzyskanie dużej różnorodności wariantów obrazów wejściowych, co potencjalnie przełoży się na lepszą generalizację modelu uczącego się.



Rysunek 2.1: Schemat działania augmentacji z użyciem klasy Augmentor. Po wczytaniu obrazka (a), następuje podział na segmenty z uwzględnieniem wybranej liczby podziałów poziomych y i liczby podziałów pionowych x oraz minimalnej liczby pikseli między podziałami (ta sama minimalna wartość zachowana w pionie i w poziomie) (b). Następnie z pewnym prawdopodobienstwiem (parametr p) każdy z segmentów jest augmentowany* (c) i wklejany z powrotem do oryginalnego obrazka (d).

2.2.1. Siatka podziału

Obraz dzielony jest na nieregularną siatkę prostokątów. Liczbę podziałów wzdłuż osi pionowej i poziomej można ustalić parametrami `vertical_splits_number` i `horizontal_splits_number`. Dodatkowo w implementacji uwzględniono losowość w określaniu dokładnego położenia linii podziału, z zachowaniem minimalnego progu odległości między sąsiadującymi liniami (`min_space_between_splits`), aby uniknąć zbyt małych lub przylegających do siebie regionów. Pozwala to uzyskać siatkę o nieregularnym układzie i różnorodnych rozmiarach prostokątów.

2.2.2. Tryby augmentacji

W początkowej implemnetacji dla każdego fragmentu obrazu (prostokąta z siatki) stosowane były z prawdopodobieństwem *p=1* lokalne augmentacje, których sposób działania zależał od wybranego trybu, określonego parametrem `mode`. Przewidziano trzy warianty:

1. `same` – dla wszystkich prostokątów stosowany jest ten sam typ augmentacji (czyli np. przesunięcie kolorów lub zmiana jasności), jednak z różną mocą przekształcenia z predefiniowanego słownika transformacji.
2. `different` – dla każdego prostokąta losowana jest dowolny typ augmentacji z predefiniowanego słownika transformacji.
3. `combine` – dla każdego prostokąta losowana jest kombinacja od jednej do trzech augmentacji ze słownika, które są następnie nakładane na siebie.

2.2.3. Augmentacje

Poniżej przedstawiono operacje zawarte w pierwotnej wersji słownika transformacji. Do implementacji transformacji wykorzystano pakiet *Albumentations*.

- **Odbicia lustrzane**: `HorizontalFlip`, `VerticalFlip` oraz ich kombinacja.
- **Rotacje**: `Rotate` z kątami 5°, 10°, 15° oraz 20°.
- **Rozmycia**: `GaussianBlur` z `blur_limit` równym $(5, 5)$, $(8, 8)$, $(11, 11)$ oraz $(14, 14)$.
- **Jasność i kontrast**: `RandomBrightnessContrast` z wartościami jasności i kontrastu odpowiednio $(0.1, 0.1)$, $(0.2, 0.2)$, $(0.3, 0.3)$ oraz $(0.4, 0.4)$.
- **Szumy**: `GaussNoise` z `std_range` równym $(0.1, 0.1)$, $(0.2, 0.2)$, $(0.3, 0.3)$ oraz $(0.4, 0.4)$.
- **Barwa**: `HueSaturationValue` z przesunięciami dokładnie $(10, 10)$, $(20, 20)$ oraz $(30, 30)$ dla barwy, nasycenia i jasności.
- **Negatyw i odcienie szarości**: `InvertImg` oraz `ToGray`.

2.3. Wizualizacja działania augmentorów

Poniżej przedstawiono przykładowe wyniki działania zaimplementowanych metod augmentacji obrazów. Wizualizacje te mają na celu zobrazowanie różnic pomiędzy klasycznym podejściem STES a naszymi wariantami opartymi na różnych strategiach modyfikowania segmentów obrazu.



(a) oryginał



(b) STES

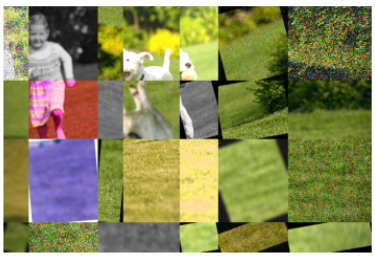
Rysunek 2.2: Działanie zaimplementowanej własnoręcznie metody STES. Po lewej, obraz oryginalny (a), po prawej obraz zaugmentowany (b).



(a) same



(b) different



(c) combine

Rysunek 2.3: Porównanie działania Augmentora dla różnych parametrów mode: *same, different i combine*. Wszystkie segmenty augmentowane z prawdopodobieństwem $p = 1$. W (a) wylosowana została augmentacja zmiany koloru, w (b) różne rodzaje augmentacji na różnych segmentach, w (c) dwie augmentacje nałożone na siebie w każdym segmencie.

Na tym etapie nasza metoda wydaje się znacznie bardziej ingerować w strukturę obrazu niż STES. Wynika to z zastosowania wysokiego poziomu prawdopodobieństwa augmentacji dla każdego segmentu ($p = 1$). W dalszej części eksperymentów parametr ten będzie stopniowo modyfikowany, co pozwoli na ocenę wpływu intensywności przekształceń na jakość wyników.

3. Pierwsze eksperymenty

3.1. Konfiguracje i struktury eksperymentów

Pierwotnie do przeprowadzenia pierwszych eksperymentów wybrano dwie architektury sieci neuronowych z rodziny *EfficientNet*: *EfficientNet-B0* oraz *EfficientNet-B7*. Dla obu modeli rozważono warianty z wykorzystaniem wstępnie wytrenowanych wag (*pretrained*) oraz bez nich (*non-pretrained*). Większość eksperymentów została przeprowadzona w oparciu o ustaloną konfigurację parametrów i hiperparametrów treningu. Zastosowano rozmiar wsadu (*batch size*) równy `64`, liczbę epok ustawioną na `20`, współczynnik uczenia `0,001` oraz funkcję straty `CrossEntropyLoss`.

W przypadku modeli *pretrained* zastosowano normalizację danych wejściowych zgodną z parametrami zbioru *ImageNet*. Obrazy zostały przeskalowane do rozmiaru 224×224, który odpowiada rozmiarom wejściowym sieci *EfficientNet*. Na wyjściu modeli dodano warstwę klasyfikacyjną dostosowaną do liczby klas poszczególnych zbiorów danych.

Eksperymenty zostały przeprowadzone na zbiorach *CIFAR-10* oraz *Fashion-MNIST*. Dane zostały podzielone na zbiór treningowy oraz testowy zgodnie z domyślnym podziałem przewidzianym w bibliotece *torchvision*. Aby umożliwić dynamiczne nakładanie własnych metod augmentacji na obrazy, przeładowano klasę `Dataset` poprzez stworzenie klasy `DatasetWrapper`. Klasa ta pozwala na stosowanie wybranych augmentacji w trakcie ładowania danych, przy czym można przekazywać parametry konfigurujące te augmentacje. Do wyboru były są metody augmentacji: metoda `STES`, autorska metoda `Augmentor` oraz brak augmentacji (`None`). W przypadku metody `Augmentor` wrapper umożliwia również dostosowanie parametrów augmentacji.

W osobnym notebooku przetestowano działanie autorskiego `Augmentatora` na obrazkach pochodzących ze zbiorów *CIFAR-10* oraz *Fashion-MNIST*. Na tym etapie eksperymentów zaobserwowano jedynie pewne podejrzane zniekształcenia kolorów w obrazach, jednak wnioskowano, że były one efektem zastosowanej normalizacji kolorów, a nie błędów w samej implementacji augmentacji lub wrappera. Poza tym, zastosowane transformacje wizualnie prezentowały się poprawnie i spełniały swoje funkcje.

Ewentualnym problemem mogło być również użycie zbyt agresywnych parametrów augmentacji. W związku z tym w kolejnych eksperyemntach postanowiono ograniczyć siłę stosowanych transformacji. Poniżej zaprezentowano przykładowe wyniki działania augmentacji przed zastosowaniem zmian.

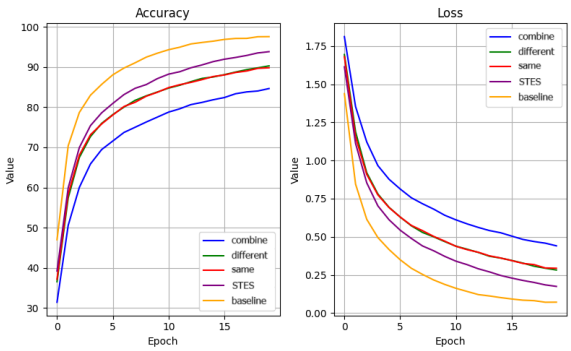


Rysunek 3.1: Przykłady zaugmentowanych z użyciem Augmentora obrazów ze zbioru *CIFAR-10* (mode *different*).

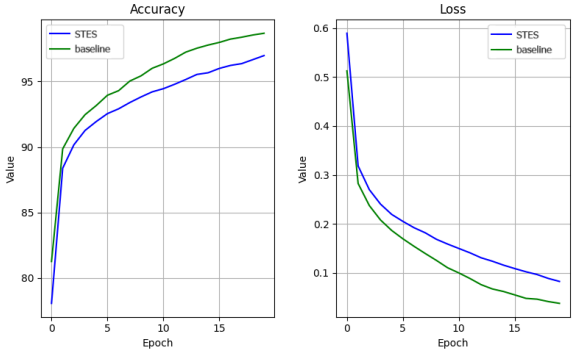


Rysunek 3.2: Przykłady zaugmentowanych z użyciem Augmentora obrazów ze zbioru *Fashion-MNIST* (mode *different*).

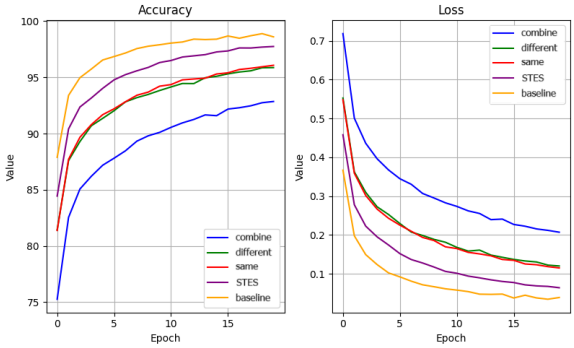
Poniżej przedstawiono przykładowe przebiegi procesu uczenia modeli w różnych konfiguracjach. Choć wizualizacje te nie pozwalają na jednoznaczne wnioskowanie o skuteczności poszczególnych metod augmentacji, mogą one stanowić pomocny kontekst przy analizie zachowania modeli w trakcie treningu — szczególnie pod kątem stabilności i tempa zbieżności. Dane te mają charakter poglądowy i pełnią rolę uzupełniającą względem głównych metryk oceny skuteczności.



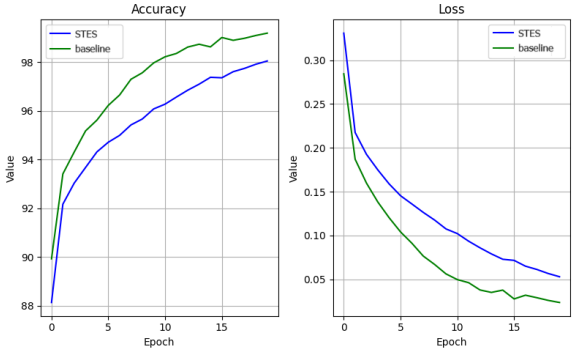
(a) *CIFAR-10* — modele bez pretrenowania



(b) *Fashion-MNIST* — modele bez pretrenowania



(c) *CIFAR-10* — modele pretrenowane na *ImageNet*



(d) *Fashion-MNIST* — modele pretrenowane na *ImageNet*

Rysunek 3.3: Wizualizacja przebiegu procesu treningowego modelu *EfficientNet-B0* na zbiorach *CIFAR-10* (lewa kolumna) oraz *Fashion-MNIST* (prawa kolumna). Dla zbioru *CIFAR-10* zaprezentowano wyniki dla wszystkich konfiguracji: modelu bazowego, wersji z metodą STES oraz z Augmentorem dla różnych mode'ów. W przypadku zbioru *Fashion-MNIST* przedstawiono jedynie wersje bazową i STES — brak pełnych danych dla Augmentora wynika z problemów z synchronizacją plików. Mimo to, na zbiorze *Fashion-MNIST* nasze metody nie wykazały wyraźnej przewagi nad bazowym podejściem, co sugeruje ograniczoną efektywność ich zastosowania w tym konkretnym przypadku.

Po przeprowadzeniu pierwszych testów zaobserwowano, że zarówno metoda STES, jak i nasz autorski Augmentor osiągały wyniki znacząco słabsze od modelu bazowego, trenowanego bez jakichkolwiek augmentacji. Rezultaty te były sprzeczne z oczekiwaniami oraz intuicją, która zakładała poprawę generalizacji modelu dzięki zastosowaniu dodatkowych przekształceń danych treningowych. W związku z tym, że uzyskane wyniki były niespójne i trudne do jednoznacznej interpretacji, nie zdecydowano się na ich szersze omówienie w tej sekcji. Zamiast tego skupiono się na analizie potencjalnych przyczyn nieprawidłowości oraz dalszym dopracowywaniu procesu przetwarzania i augmentacji danych, co opisano w kolejnych częściach raportu.

4. Eksperymenty - ciąg dalszy

4.1. Konfiguracje i struktury kolejnych eksperymentów

W kolejnych eksperymentach wprowadzono kilka istotnych zmian w konfiguracji. Dotychczasowy zbiór treningowy został podzielony na nowy zbiór treningowy oraz zbiór walidacyjny, co umożliwiło dokładniejsze monitorowanie procesu uczenia i ocenę działania modelu na danych niewidzianych podczas treningu.
Ponadto, zgodnie z obserwacjami opisanymi w poprzedniej sekcji, ograniczono intensywność stosowanych augmentacji. Nowa wersja słownika augmentacji zawierała następujące zakresy:

- **Rotacje**: od $\pm 5°$ do $\pm 15°$
- **Rozmycia**: `blur_limit` od $(5, 5)$ do $(14, 14)$
- **Jasność i kontrast**: `brightnes_limit` oraz `contrast_limit` z zakresu od $(0.03, 0.03)$ do $(0.1, 0.1)$
- **Szumy**: `std_range` od $(0.01, 0.01)$ do $(0.05, 0.05)$
- **Odbicia**: odbicie poziome, pionowe oraz ich kombinacja

W celu zweryfikowania przyczyn braku poprawy wyników — oraz faktu, że najlepsze rezultaty osiągane były przez model bazowy (bez augmentacji) — przeprowadzono dodatkowe eksperymenty. W ich ramach przetestowano wyłącznie rotację całego obrazu, a także augmentacje zastosowane w metodzie `Augmentor` (w wariantach `same` oraz `combine`), jednak przy liczbie podziałów obrazu równej `0` w obu kierunkach, co odpowiadało klasycznej augmentacji całego obrazu bez dzielenia go na segmenty.

4.2. Wyniki drugiej serii eksperymentów

W trakcie kolejnych eksperymentów zaobserwowano nietypowy błąd, który występował wyłącznie dla modeli pretrenowanych. Od drugiej epoki następował gwałtowny wzrost wartości funkcji straty (loss), a dokładność klasyfikacji (accuracy) utrzymywała się na bardzo niskim poziomie (około 10%). Co istotne, zjawisko to występowało niezależnie od zastosowanej metody augmentacji danych.

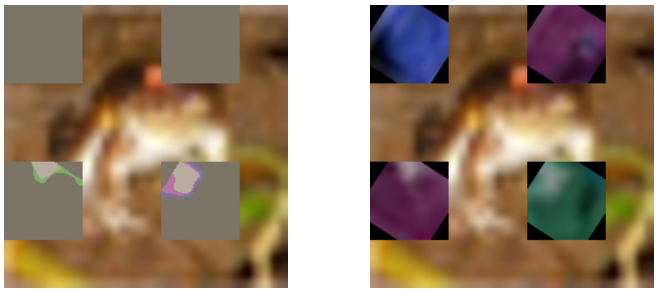
Początkowo podejrzewano, że problem może wynikać z niestabilności gradientów lub eksplozji wag w początkowej fazie uczenia. W związku z tym, wprowadzono modyfikację polegającą na zamrożeniu wszystkich warstw modelu (oprócz warstwy klasyfikacyjnej) przez pierwsze siedem epok treningu. Dopiero po tym okresie odblokowano możliwość aktualizacji wag w całej sieci. Takie rozwiązanie przyniosło zauważalną poprawę — wartość funkcji straty utrzymywała się na stabilnym poziomie, a dokładność wzrosła. Co jednak zaskakujące, po cofnięciu tej zmiany, opisany problem nie wystąpił ponownie, co pozostawia jego przyczynę niejasną.

Podobna sytuacja miała miejsce przy uruchomieniu eksperymentu dla metody `STES` tym razem w wariancie niepretrenowanym. Również w tym przypadku model wykazywał nietypowe, losowe zachowanie — ekstremalny wzrost funkcji straty i bardzo niska skuteczność klasyfikacji. Jednak ponowne uruchomienie tej samej konfiguracji nie skutkowało powtórzeniem błędu, co może sugerować losowy charakter problemu (np. niefortunnie zainicjalizowane wagi). Ze względu na ograniczenia czasowe oraz konieczność realizacji dalszych etapów projektu, nie przeprowadzono dodatkowych prób, które pozwoliłyby jednoznacznie ocenić, na ile obie te sytuacje miały charakter przypadkowy.

4.3. Poprawa błędów technicznych w augmentacji obrazów

Podczas kolejnych testów udało się zidentyfikować istotny błąd techniczny w procesie przetwarzania danych. Obrazy były wczytywane jako tensory z zakresem wartości `[0, 1]`, a następnie konwertowane do formatu `NumPy` bez przeskalowania do przedziału `[0, 255]`, który wymagany jest przez bibliotekę `Albumentations`. W rezultacie, operacje augmentacyjne były wykonywane na nieprawidłowym zakresie intensywności pikseli, co prowadziło do zniekształceń obrazu — szczególnie w przypadku transformacji opartych na manipulacji kolorami (np. zmiana jasności, kontrastu czy barwy).

Co istotne, problem ten nie został wykryty podczas początkowych testów przeprowadzanych na pojedynczych obrazach, ponieważ do ich wizualizacji użyto osobnego kodu, w którym zastosowano prawidłowe przeskalowanie danych wejściowych. Błąd pojawił się dopiero na etapie implementacji klasy `DatasetWrapper`, gdzie wspomniana konwersja została pominięta. Warto podkreślić, że augmentacje niezależne od wartości intensywności kolorów, takie jak rotacje, odbicia lustrzane czy rozmycia, działały poprawnie i nie były dotknięte tym błędem.



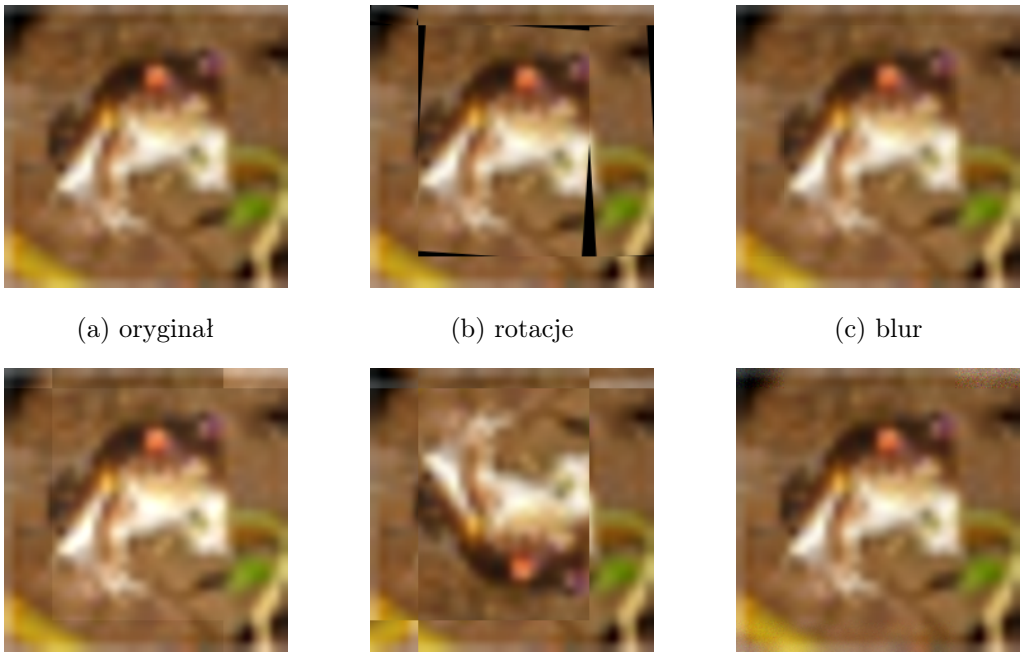
(a) przed (b) po

Rysunek 4.1: Main caption for the two side-by-side images

Rysunek 4.2: Przykład błędu w kolorystycznych augmentacjach, który występował przy użyciu augmentorów. Po lewej stronie widoczny jest efekt przed poprawką – augmentacja działała bardziej jak maskowanie, zamiast realistycznie modyfikować kolory. Po prawej stronie pokazano rezultat po naprawieniu błędu, gdzie augmentacja funkcjonuje zgodnie z założeniem.

4.4. Kontynuacja eksperymentów

Po zidentyfikowaniu i naprawieniu istotnego błędu związanego z nieprawidłowym przeskalowaniem obrazów, zdecydowano się – ze względu na ograniczenia czasowe oraz dostępne zasoby obliczeniowe – na uproszczenie ostatecznej konfiguracji eksperymentów. Od tej pory eksperymenty były prowadzone tylko na zbiorze *CIFAR-10* z użyciem mniejszego modelu *MobileNetV2*. W finalnej fazie testów zastosowano jednocześnie tylko jedną metodę augmentacji podczas treningu, co określono jako tryb `same`. Podejście to miało na celu ograniczenie liczby kombinacji do przetestowania oraz umożliwienie bardziej przejrzystej analizy wpływu pojedynczych technik augmentacyjnych na jakość działania modelu. Poniżej zostały przedstawione przykłady różnych augmentacji zastosowanych przy użyciu Augmentora w trybie `same` na wybranym obrazku ze zbioru *CIFAR-10*.



(a) oryginał (b) rotacje (c) blur

(d) jasność (e) odbicia (f) szum

Rysunek 4.3: Przykłady różnych augmentacji zastosowanych przy użyciu Augmentora, mode `same`, na obrazku z *CIFAR-10*. Przedstawione obrazy zostały wygenerowane z podziałem obrazka dla $x = y = 3$ oraz z minimalnym odstępem między segmentami równym 10 pikseli. Augmentacje zostały wykorzystane w drugiej fazie eksperymentów i obejmują rotację, rozmycie, zmianę jasności, odbicia lustrzane oraz dodanie szumu.

Przetestowano kolejno następujące techniki augmentacji: `Blur`, `Brightness`, `Flip`, `Noise` oraz `Rotate`. Dla każdej z powyższych metod augmentacji przeanalizowano wpływ różnych konfiguracji podziału obrazu — tj. liczby podziałów na osi `x` i osi `y`. Uwzględniono następujące warianty podziałów:

- 0×0 — brak podziału (augmentacja stosowana na całym obrazie)
- 1×1
- 1×3
- 2×2
- 3×1
- 3×3
- 9×9

Dla każdej kombinacji — konkretna metoda augmentacji oraz wariant podziału obrazu — uruchomiono proces treningowy sześciokrotnie, aby uzyskać uśrednione wyniki i ograniczyć wpływ losowości.

Dodatkowo, w celach porównawczych, wykonano sześć uruchomień dla modelu bez augmentacji oraz sześć dla metody `STES`.

Tabela 4.2 przedstawia uśrednione wyniki dla każdego z wariantów na podstawie sześciu przebiegów dla prawdopodobieństwa augmentacji segmentu $p = 0.8$. Rysunek 4.4 przedstawia wykresy obrazujące wyniki na zbiorach testowych dla poszczególnych augumentacji i metryk.

Tryb	**Podział**	**Loss**	**Accuracy**	**Recall**	**Precision**	**F1**
baseline	0x0	0.614 ± 0.035	84.095 ± 0.546	0.841 ± 0.005	0.844 ± 0.006	0.841 ± 0.006
stes	0x0	0.424 ± 0.019	86.400 ± 0.530	0.864 ± 0.005	0.866 ± 0.003	0.864 ± 0.005
blur	0x0	0.610 ± 0.033	84.375 ± 0.375	0.844 ± 0.004	0.847 ± 0.003	0.843 ± 0.004
	1x1	0.603 ± 0.031	**84.630 ± 0.545**	0.846 ± 0.005	0.849 ± 0.007	0.846 ± 0.006
	1x3	0.591 ± 0.050	84.332 ± 0.673	0.843 ± 0.007	0.845 ± 0.006	0.843 ± 0.006
	2x2	0.639 ± 0.024	84.397 ± 0.437	0.844 ± 0.004	0.846 ± 0.005	0.843 ± 0.005
	3x1	0.631 ± 0.062	84.062 ± 0.531	0.841 ± 0.005	0.843 ± 0.005	0.840 ± 0.006
	3x3	0.627 ± 0.027	84.227 ± 0.386	0.842 ± 0.004	0.844 ± 0.004	0.841 ± 0.005
	9x9	0.595 ± 0.028	84.527 ± 0.423	0.845 ± 0.004	0.847 ± 0.005	0.845 ± 0.004
brightness	0x0	0.645 ± 0.016	84.030 ± 0.364	0.840 ± 0.004	0.844 ± 0.004	0.840 ± 0.004
	1x1	0.612 ± 0.055	84.355 ± 0.374	0.844 ± 0.004	0.847 ± 0.004	0.843 ± 0.004
	1x3	0.602 ± 0.031	84.812 ± 0.642	0.848 ± 0.006	0.850 ± 0.006	0.848 ± 0.007
	2x2	0.579 ± 0.030	84.553 ± 0.352	0.846 ± 0.004	0.848 ± 0.005	0.845 ± 0.004
	3x1	0.577 ± 0.039	84.810 ± 0.272	0.848 ± 0.003	0.849 ± 0.002	0.848 ± 0.003
	3x3	0.585 ± 0.029	**84.945 ± 0.399**	0.849 ± 0.004	0.851 ± 0.003	0.849 ± 0.004
	9x9	0.573 ± 0.025	84.847 ± 0.629	0.848 ± 0.006	0.851 ± 0.006	0.848 ± 0.006
flip	0x0	0.440 ± 0.009	85.275 ± 0.233	0.853 ± 0.002	0.856 ± 0.003	0.852 ± 0.002
	1x1	0.447 ± 0.022	85.290 ± 0.517	0.853 ± 0.005	0.857 ± 0.003	0.853 ± 0.004
	1x3	0.449 ± 0.011	84.892 ± 0.431	0.849 ± 0.004	0.852 ± 0.004	0.849 ± 0.004
	2x2	0.423 ± 0.012	85.828 ± 0.717	0.858 ± 0.007	0.860 ± 0.008	0.858 ± 0.008
	3x1	0.406 ± 0.014	**86.315 ± 0.473**	0.863 ± 0.005	0.866 ± 0.004	0.863 ± 0.004
	3x3	0.445 ± 0.017	84.662 ± 0.602	0.847 ± 0.006	0.851 ± 0.005	0.846 ± 0.008
	9x9	0.488 ± 0.020	83.833 ± 0.403	0.838 ± 0.004	0.844 ± 0.002	0.839 ± 0.003
noise	0x0	0.657 ± 0.035	83.810 ± 0.218	0.838 ± 0.002	0.843 ± 0.003	0.838 ± 0.002
	1x1	0.634 ± 0.028	84.292 ± 0.447	0.843 ± 0.004	0.844 ± 0.005	0.842 ± 0.005
	1x3	0.632 ± 0.016	84.047 ± 0.487	0.840 ± 0.005	0.843 ± 0.005	0.841 ± 0.005
	2x2	0.648 ± 0.032	83.888 ± 0.345	0.839 ± 0.003	0.842 ± 0.003	0.839 ± 0.003

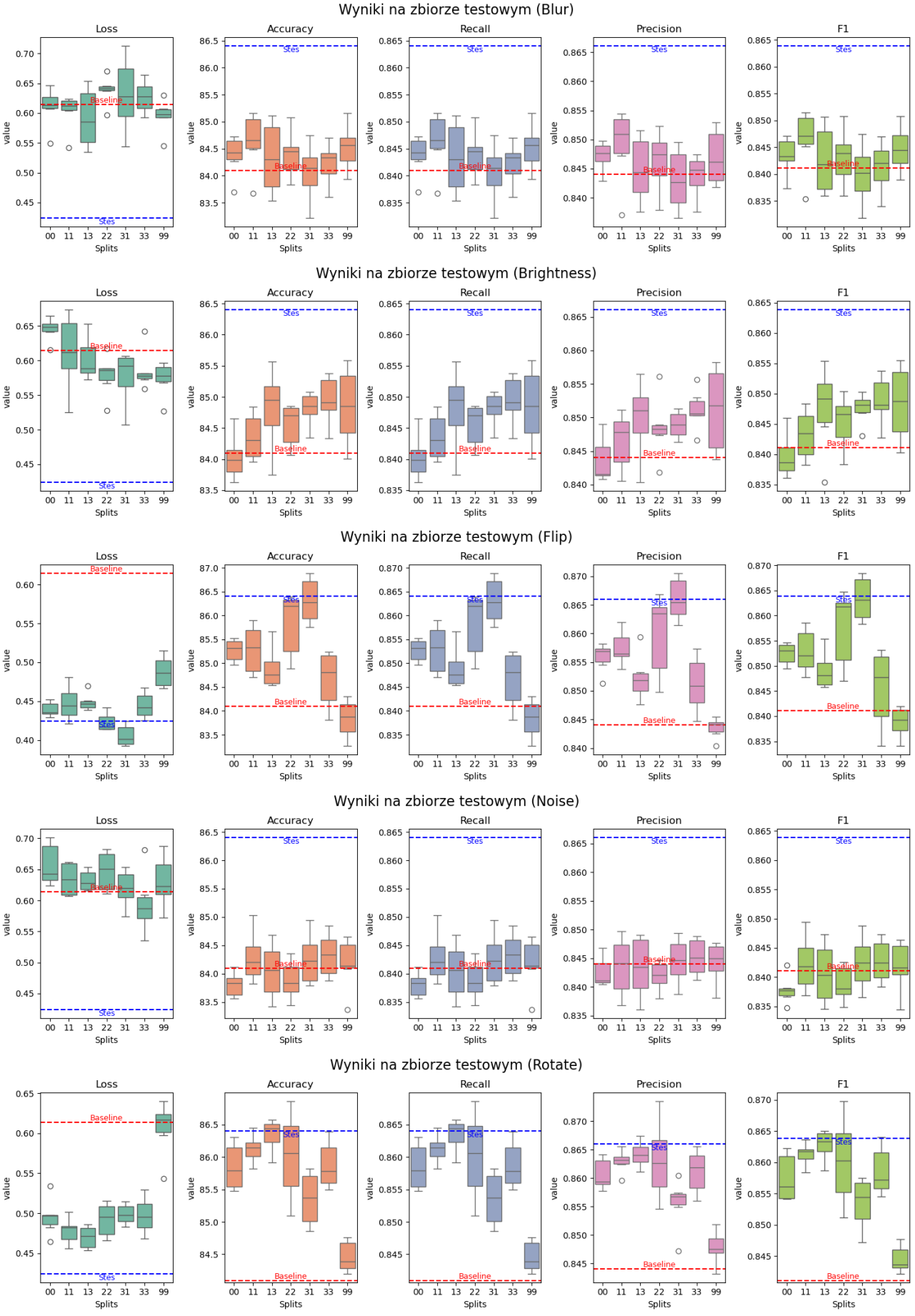
ciąg dalszy na następnej stronie

Tryb	Podział	Loss	Accuracy	Recall	Precision	F1
	3x1	0.620 ± 0.029	84.260 ± 0.454	0.843 ± 0.005	0.845 ± 0.004	0.842 ± 0.005
	3x3	0.595 ± 0.049	**84.330 ± 0.395**	0.843 ± 0.004	0.845 ± 0.003	0.843 ± 0.004
	9x9	0.630 ± 0.042	84.172 ± 0.469	0.842 ± 0.005	0.844 ± 0.004	0.842 ± 0.004
rotate	0x0	0.495 ± 0.023	85.850 ± 0.362	0.859 ± 0.004	0.861 ± 0.003	0.857 ± 0.004
	1x1	0.478 ± 0.016	86.130 ± 0.217	0.861 ± 0.002	0.863 ± 0.002	0.861 ± 0.002
	1x3	0.470 ± 0.014	**86.345 ± 0.250**	0.863 ± 0.003	0.864 ± 0.002	0.863 ± 0.002
	2x2	0.492 ± 0.021	86.013 ± 0.671	0.860 ± 0.007	0.863 ± 0.007	0.860 ± 0.007
	3x1	0.499 ± 0.013	85.355 ± 0.411	0.854 ± 0.004	0.856 ± 0.004	0.854 ± 0.004
	3x3	0.497 ± 0.023	85.878 ± 0.364	0.859 ± 0.004	0.861 ± 0.004	0.859 ± 0.004
	9x9	0.607 ± 0.034	84.458 ± 0.243	0.845 ± 0.002	0.848 ± 0.003	0.844 ± 0.002

Tabela 4.2: Porównanie wyników na zbiorze testowym dla drugiej serii eksperymentów.
Prawdopodobieństwo augmentacji segmentu $p = 0.8$

Zarówno w tabeli powyżej jak i na rysunku poniżej można zauważyć, że najlepsze średnie wyniki osiągnięto dla augmentacji typu *flip* i *rotate*, co jest zgodne z oczekiwaniami. Mimo to, ich przewaga nad modelem bazowym była niewielka, a znaczną poprawę przynosiły jedynie pojedyncze uruchomienia. Może to świadczyć o nadmiernej ingerencji w dane — szczególnie przy wysokim prawdopodobieństwie augmentacji $p = 0.8$, które powodowało, że modyfikowana była zbyt duża część obrazu.

Dla bardziej agresywnych ustawień, takich jak podział obrazu na dużą liczbę małych segmentów (np. 9×9), transformacje geometryczne miały wyraźnie negatywny wpływ. Jest to prawdopodobnie spowodowane powstawaniem czarnych obszarów po rotacjach i odbiciach, zwłaszcza w przypadku wąskich, prostokątnych segmentów. Z kolei metody takie jak dodawanie szumu czy rozmycie nie przyniosły wyraźnych korzyści, a ich wyniki były porównywalne z baseline'em.



Rysunek 4.4: Porównanie wyników na zbiorze testowym dla drugiej serii eksperymentów. Prawdopodobieństwo augmentacji segmentu $p = 0.8$.

UWAGA: Wartości na osi x reprezenują liczbę podziałów obrazka zapisaną w formacie xy, gdzie x - liczba podziałów pionowych, y - liczba podziałów poziomych.

4.4.1. Modyfikacja parametru p

W celu dokładniejszego porównania z metodą STES, która zakłada augmentację około 25% obrazu, zdecydowano się na zmniejszenie prawdopodobieństwa augmentacji każdego segmentu w naszej metodzie do wartości $p = 0.25$. Dzięki temu możliwa była bardziej bezpośrednia ocena wpływu modyfikacji na jakość wyników. Uzyskane rezultaty zostały przedstawione w poniższej tabeli.

Tryb	Podział	Loss	Accuracy	Recall	Precision	F1
baseline	0x0	0.608 ± 0.049	84.120 ± 0.371	0.841 ± 0.004	0.843 ± 0.004	0.841 ± 0.004
stes	0x0	0.425 ± 0.022	86.358 ± 0.797	0.864 ± 0.008	0.867 ± 0.007	0.864 ± 0.008
blur	0x0	0.594 ± 0.017	84.497 ± 0.342	0.845 ± 0.003	0.846 ± 0.005	0.844 ± 0.004
	1x1	0.603 ± 0.024	84.303 ± 0.396	0.843 ± 0.004	0.846 ± 0.004	0.843 ± 0.004
	1x3	0.618 ± 0.024	84.435 ± 0.422	0.844 ± 0.004	0.847 ± 0.005	0.844 ± 0.005
	2x2	0.614 ± 0.021	84.632 ± 0.327	0.846 ± 0.003	0.848 ± 0.003	0.846 ± 0.003
	3x1	0.593 ± 0.030	**84.742 ± 0.300**	0.847 ± 0.003	0.849 ± 0.003	0.847 ± 0.003
	3x3	0.588 ± 0.047	84.433 ± 0.407	0.844 ± 0.004	0.846 ± 0.004	0.843 ± 0.005
	9x9	0.616 ± 0.038	84.397 ± 0.452	0.844 ± 0.005	0.844 ± 0.005	0.843 ± 0.005
brightness	0x0	0.610 ± 0.034	84.663 ± 0.654	0.847 ± 0.007	0.850 ± 0.006	0.846 ± 0.007
	1x1	0.623 ± 0.028	84.528 ± 0.267	0.845 ± 0.003	0.847 ± 0.002	0.845 ± 0.003
	1x3	0.565 ± 0.029	84.580 ± 0.734	0.846 ± 0.007	0.847 ± 0.006	0.846 ± 0.007
	2x2	0.599 ± 0.016	84.743 ± 0.349	0.847 ± 0.003	0.850 ± 0.003	0.848 ± 0.003
	3x1	0.596 ± 0.029	84.282 ± 0.393	0.843 ± 0.004	0.845 ± 0.004	0.843 ± 0.004
	3x3	0.594 ± 0.033	84.813 ± 0.695	0.848 ± 0.007	0.854 ± 0.006	0.848 ± 0.007
	9x9	0.577 ± 0.020	**84.987 ± 0.373**	0.850 ± 0.004	0.851 ± 0.004	0.849 ± 0.004
flip	0x0	0.440 ± 0.018	85.567 ± 0.337	0.856 ± 0.003	0.858 ± 0.003	0.855 ± 0.004
	1x1	0.444 ± 0.018	85.483 ± 0.564	0.855 ± 0.006	0.858 ± 0.004	0.854 ± 0.006
	1x3	0.451 ± 0.017	84.730 ± 0.392	0.847 ± 0.004	0.850 ± 0.003	0.847 ± 0.004
	2x2	0.430 ± 0.016	85.493 ± 0.551	0.855 ± 0.006	0.859 ± 0.004	0.854 ± 0.006
	3x1	0.411 ± 0.011	**86.088 ± 0.375**	0.861 ± 0.004	0.863 ± 0.003	0.860 ± 0.004
	3x3	0.443 ± 0.016	84.727 ± 0.659	0.847 ± 0.007	0.851 ± 0.006	0.847 ± 0.007
	9x9	0.487 ± 0.007	83.840 ± 0.313	0.838 ± 0.003	0.842 ± 0.002	0.838 ± 0.003

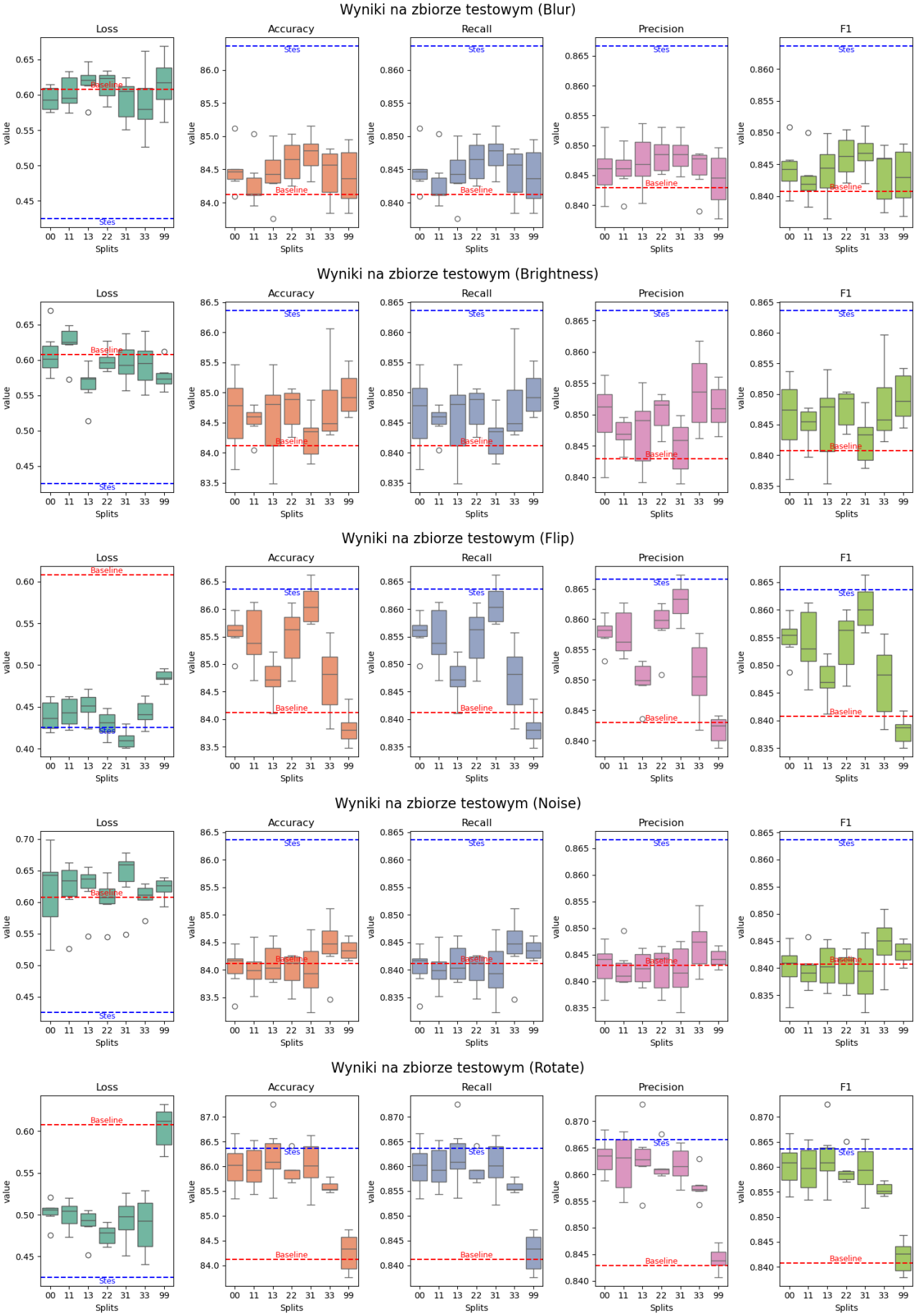
ciąg dalszy na następnej stronie

Tryb	Podział	Loss	Accuracy	Recall	Precision	F1
noise	0x0	0.619 ± 0.065	84.035 ± 0.393	0.840 ± 0.004	0.843 ± 0.004	0.840 ± 0.004
	1x1	0.619 ± 0.050	84.013 ± 0.367	0.840 ± 0.004	0.843 ± 0.004	0.840 ± 0.003
	1x3	0.623 ± 0.040	84.125 ± 0.358	0.841 ± 0.004	0.843 ± 0.003	0.840 ± 0.004
	2x2	0.605 ± 0.035	83.995 ± 0.322	0.840 ± 0.003	0.842 ± 0.004	0.840 ± 0.004
	3x1	0.639 ± 0.048	83.982 ± 0.551	0.840 ± 0.006	0.842 ± 0.005	0.839 ± 0.006
	3x3	0.608 ± 0.021	**84.427 ± 0.561**	0.844 ± 0.006	0.847 ± 0.005	0.844 ± 0.005
	9x9	0.622 ± 0.017	84.370 ± 0.184	0.844 ± 0.002	0.844 ± 0.002	0.843 ± 0.002
rotate	0x0	0.503 ± 0.015	86.003 ± 0.476	0.860 ± 0.005	0.863 ± 0.003	0.860 ± 0.005
	1x1	0.500 ± 0.017	85.980 ± 0.436	0.860 ± 0.004	0.862 ± 0.006	0.860 ± 0.005
	1x3	0.489 ± 0.019	**86.213 ± 0.640**	0.862 ± 0.006	0.863 ± 0.006	0.862 ± 0.006
	2x2	0.476 ± 0.012	85.925 ± 0.267	0.859 ± 0.003	0.862 ± 0.003	0.859 ± 0.003
	3x1	0.494 ± 0.027	86.017 ± 0.522	0.860 ± 0.005	0.862 ± 0.003	0.859 ± 0.005
	3x3	0.488 ± 0.035	85.593 ± 0.120	0.856 ± 0.001	0.858 ± 0.003	0.856 ± 0.001
	9x9	0.605 ± 0.026	84.270 ± 0.404	0.843 ± 0.004	0.844 ± 0.002	0.842 ± 0.003

Tabela 4.4: Porównanie wyników na zbiorze testowym dla drugiej serii eksperymentów.
Prawdopodobieństwo augmentacji segmentu $p = 0.25$

Wartości metryk uzyskane po zmniejszeniu prawdopodobieństwa augmentacji do $p = 0.25$ nadal pozostają bliższe wartościom bazowym niż wynikom osiągniętym przez metodę STES. Najlepiej w dalszym ciągu wypadają augmentacje typu *flip* oraz *rotate*.

W przypadku *flip*, zarówno w tym, jak i w poprzednim eksperymencie, zauważono największą wrażliwość na liczbę podziałów - wyniki wykazują dużą zmienność w zależności od tego parametru. Dla *blur* i *noise* wpływ liczby podziałów był znacznie mniej widoczny, co prawdopodobnie wynika z faktu, że przekształcenia te mniej ingerują w globalną strukturę obrazu. Mimo zmniejszenia intensywności augmentacji, metoda STES wciąż osiąga lepsze wyniki.

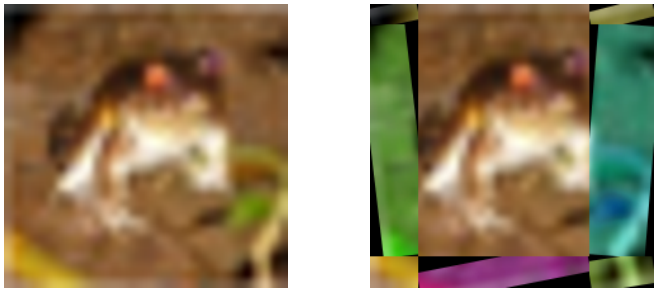


Rysunek 4.5: Porównanie wyników na zbiorze testowym dla drugiej serii eksperymentów. Prawdopodobieństwo augmentacji segmentu $p = 0.25$.

UWAGA: Wartości na osi x reprezenują liczbę podziałów obrazka zapisaną w formacie *xy*, gdzie *x* - liczba podziałów pionowych, *y* - liczba podziałów poziomych.

4.5. Trzecia faza eksperymentów — augmentacje złożone

Trzecia faza eksperymentów dotyczyła testowania złożonych kombinacji augmentacji. Wariant oznaczony jako *combine* został specjalnie zmodyfikowany tak, aby jak najwierniej odwzorować typ przekształceń stosowanych w metodzie STES, przy jednoczesnym zachowaniu naszego podejścia segmentacyjnego. Jest to jedyny przypadek, w którym w jednym segmencie stosowano sekwencyjnie więcej niż jedną transformację — najpierw zmianę barwy, a następnie rotację. Przykład działania metody został przedstawiony na rysunku poniżej.



(a) oryginał (b) combine

Rysunek 4.6: Przykładowy obraz ze zbioru *CIFAR-10* oraz obraz po zastosowaniu Augmentora z nowym ustalonym trybem *combine*: losowe rotacje z zakresu (-15,15) i zmiany kolorystyczne. Zestaw augmentacji nałożony na dany segment z prawdopodbieństwem *p=0.25*

Dla pozostałych wariantów (*flip-rotate*, *flip-brightness*, *rotate-brightness*, *flip-rotate-brightness*) zdecydowano się połączyć augmentacje, które w poprzednich fazach osiągały najlepsze wyniki w ramach naszej metody. W każdej iteracji na wybranym segmencie stosowano jedną losowo dobraną transformację spośród zadanej kombinacji, co miało na celu sprawdzenie, czy zestawienia skutecznych przekształceń mogą przynieść lepsze rezultaty niż każda z nich osobno.

Przykładowe efekty działania tych wariantów przedstawiono na poniższych ilustracjach.



(a) oryginał (b) flip-rotate-brightness

Rysunek 4.7: Przykładowy obraz z *CIFAR-10* i działanie zestawu augmentacji *flip-rotate-brighness* z użyciem Augmentor stosowanego podczas trzeciej fazy eksperymentów



(a) flip-rotate (b) flip-brightness (c) rotate-brighness

Rysunek 4.8: Przykładowe działanie metody Augmentor dla zestawów augmentacji *flip-rotate*, *flip-brightness* i *rotate-brightness* użytych podczas trzeciej fazy eksperymentów

4.5.1. Wyniki trzeciej serii eksperymentów

Poniższa tabela przedstawia uśrednione wyniki oraz odchylenia standardowe dla każdej z testowanych kombinacji augmentacji w trzeciej fazie eksperymentów. Zestawiono je dla różnych typów podziałów obrazu. Celem tej serii było sprawdzenie, czy zestawienia wcześniej skutecznych transformacji, a także dokładniejsza symulacja działania STES, mogą poprawić jakość predykcji w naszym podejściu.

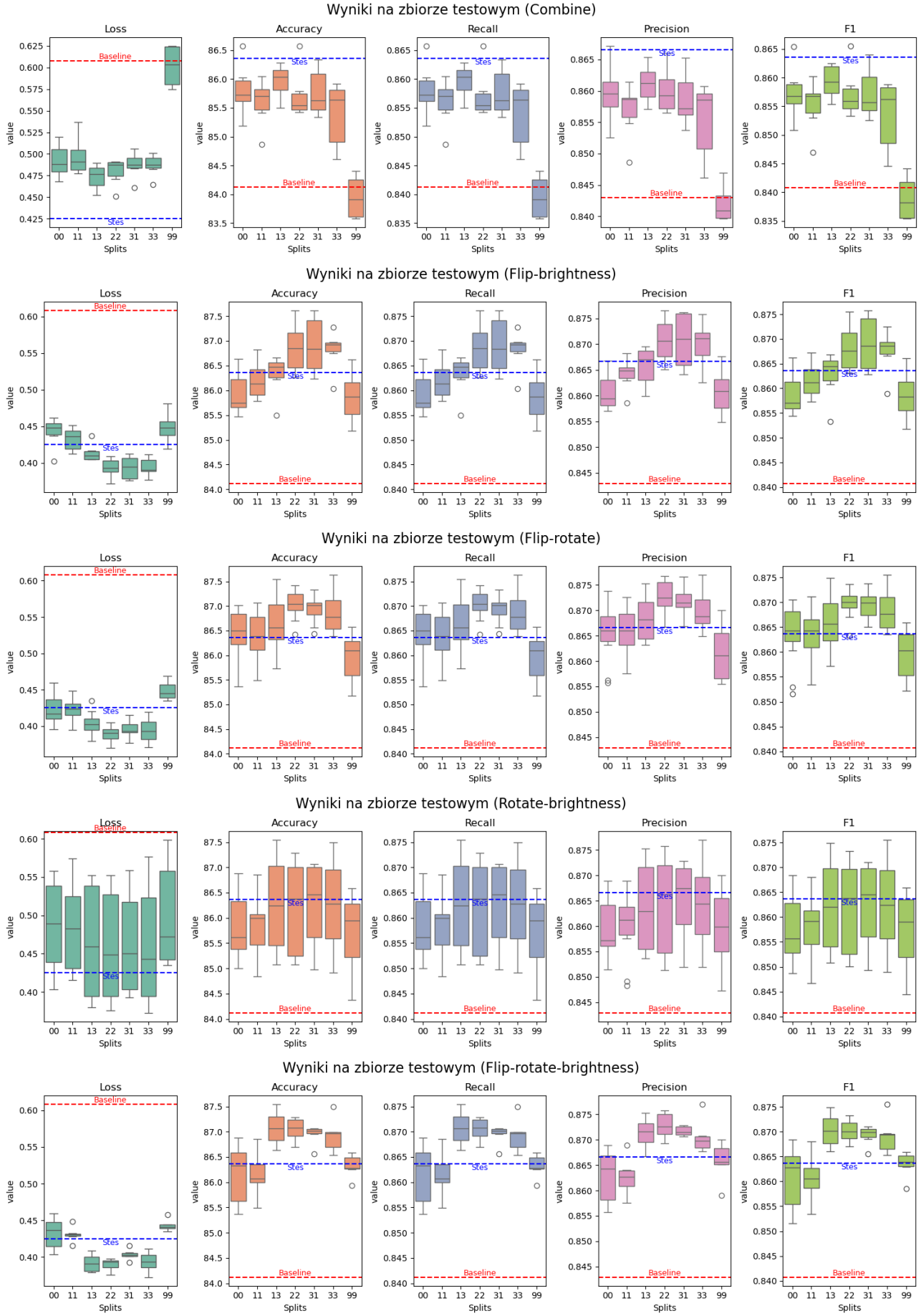
Tryb	Podział	Loss	Accuracy	Recall	Precision	F1
baseline	0x0	0.608 ± 0.049	84.120 ± 0.371	0.841 ± 0.004	0.843 ± 0.004	0.841 ± 0.004
stes	0x0	0.425 ± 0.022	86.358 ± 0.797	0.864 ± 0.008	0.867 ± 0.007	0.864 ± 0.008
combine	0x0	0.492 ± 0.019	85.808 ± 0.465	0.858 ± 0.005	0.860 ± 0.005	0.857 ± 0.005
	1x1	0.497 ± 0.022	85.597 ± 0.415	0.856 ± 0.004	0.857 ± 0.005	0.855 ± 0.005
	1x3	0.473 ± 0.014	**85.963 ± 0.290**	0.860 ± 0.003	0.861 ± 0.003	0.859 ± 0.003
	2x2	0.480 ± 0.016	85.720 ± 0.438	0.857 ± 0.004	0.860 ± 0.004	0.857 ± 0.004

ciąg dalszy na następnej stronie

Tryb	Podział	Loss	Accuracy	Recall	Precision	F1
	3x1	0.487 ± 0.015	85.767 ± 0.418	0.858 ± 0.004	0.859 ± 0.004	0.857 ± 0.004
	3x3	0.487 ± 0.013	85.392 ± 0.584	0.854 ± 0.006	0.855 ± 0.006	0.853 ± 0.006
	9x9	0.602 ± 0.027	83.953 ± 0.415	0.840 ± 0.004	0.842 ± 0.003	0.839 ± 0.004
flip-brightness	0x0	0.442 ± 0.021	85.932 ± 0.457	0.859 ± 0.005	0.861 ± 0.004	0.859 ± 0.005
	1x1	0.433 ± 0.016	86.205 ± 0.400	0.862 ± 0.004	0.864 ± 0.003	0.862 ± 0.004
	1x3	0.414 ± 0.012	86.312 ± 0.431	0.863 ± 0.004	0.866 ± 0.004	0.863 ± 0.005
	2x2	0.394 ± 0.014	86.858 ± 0.517	0.869 ± 0.005	0.871 ± 0.005	0.868 ± 0.005
	3x1	0.394 ± 0.017	**86.903 ± 0.599**	0.869 ± 0.006	0.871 ± 0.006	0.869 ± 0.006
	3x3	0.395 ± 0.013	86.812 ± 0.419	0.868 ± 0.004	0.870 ± 0.005	0.867 ± 0.005
	9x9	0.448 ± 0.021	85.868 ± 0.531	0.859 ± 0.005	0.861 ± 0.005	0.859 ± 0.005
flip-rotate-brightness	0x0	0.433 ± 0.022	86.160 ± 0.640	0.862 ± 0.006	0.863 ± 0.006	0.861 ± 0.007
	1x1	0.431 ± 0.011	86.153 ± 0.461	0.862 ± 0.005	0.863 ± 0.004	0.861 ± 0.005
	1x3	0.392 ± 0.012	**87.073 ± 0.343**	0.871 ± 0.003	0.871 ± 0.003	0.870 ± 0.003
	2x2	0.390 ± 0.008	87.048 ± 0.238	0.870 ± 0.002	0.873 ± 0.003	0.870 ± 0.002
	3x1	0.404 ± 0.007	86.945 ± 0.194	0.869 ± 0.002	0.872 ± 0.001	0.869 ± 0.002
	3x3	0.393 ± 0.014	86.925 ± 0.339	0.869 ± 0.003	0.871 ± 0.003	0.869 ± 0.004
	9x9	0.443 ± 0.008	86.313 ± 0.236	0.863 ± 0.002	0.866 ± 0.004	0.863 ± 0.003
flip-rotate	0x0	0.415 ± 0.013	86.637 ± 0.359	0.866 ± 0.004	0.868 ± 0.004	0.866 ± 0.004
	1x1	0.414 ± 0.013	86.672 ± 0.363	0.867 ± 0.004	0.869 ± 0.003	0.867 ± 0.003
	1x3	0.414 ± 0.013	86.193 ± 0.316	0.862 ± 0.003	0.865 ± 0.002	0.861 ± 0.003
	2x2	0.387 ± 0.012	**87.020 ± 0.330**	0.870 ± 0.003	0.873 ± 0.004	0.870 ± 0.003
	3x1	0.388 ± 0.006	86.918 ± 0.336	0.869 ± 0.003	0.872 ± 0.004	0.869 ± 0.004
	3x3	0.394 ± 0.020	86.848 ± 0.570	0.868 ± 0.006	0.870 ± 0.005	0.868 ± 0.006
	9x9	0.454 ± 0.012	85.622 ± 0.350	0.856 ± 0.003	0.858 ± 0.003	0.856 ± 0.003
rotate-brightness	0x0	0.540 ± 0.017	85.442 ± 0.267	0.854 ± 0.003	0.856 ± 0.003	0.854 ± 0.003
	1x1	0.534 ± 0.022	85.447 ± 0.531	0.854 ± 0.005	0.857 ± 0.007	0.855 ± 0.006
	1x3	0.536 ± 0.017	85.460 ± 0.299	0.855 ± 0.003	0.856 ± 0.002	0.854 ± 0.003
	2x2	0.528 ± 0.019	85.435 ± 0.385	0.854 ± 0.004	0.856 ± 0.004	0.854 ± 0.004
	3x1	0.527 ± 0.033	**85.657 ± 0.466**	0.857 ± 0.005	0.859 ± 0.004	0.856 ± 0.005
	3x3	0.527 ± 0.038	85.467 ± 0.442	0.855 ± 0.004	0.857 ± 0.003	0.855 ± 0.004
	9x9	0.554 ± 0.038	85.150 ± 0.608	0.852 ± 0.006	0.854 ± 0.005	0.851 ± 0.005

Tabela 4.6: Porównanie wyników na zbiorze testowym dla trzeciej serii eksperymentów - różne augmentacje. Prawdopodobieństwo augmentacji segmentu p = 0.25

Wyniki trzeciej fazy eksperymentów pokazują, że odpowiednie połączenie różnych rodzajów augmentacji oraz dobrze dobrana liczba podziałów obrazu pozwalają osiągnąć rezultaty lepsze niż metoda STES. Mimo że tryb *combine*, zaprojektowany jako bezpośredni odpowiednik STES w naszym podejściu, wypadł słabiej, inne konfiguracje okazały się bardziej efektywne, co można zaubserwować również w rysunku poniżej.



Rysunek 4.9: Porównanie wyników na zbiorze testowym dla trzeciej serii eksperymentów - różne augmentacje. Prawdopodobieństwo augmentacji segmentu $p = 0.25$.

UWAGA: Wartości na osi x reprezenują liczbę podziałów obrazka zapisaną w formacie *xy*, gdzie *x* - liczba podziałów pionowych, *y* - liczba podziałów poziomych.

Najlepsze wyniki uzyskano dla połączeń *flip-rotate* oraz *flip-brightness* przy podziałach takich jak 2×2, 3×1 oraz 3×3. W przypadku zestawu *flip-rotate-brightness* dobrze sprawdził się również podział 1×3, co pokazuje, że właściwe dopasowanie strategii może mieć znaczący wpływ na jakość wyników.

Choć wzrost wartości metryk nie był spektakularny, to jednak konsekwentnie wskazuje na potencjał naszego podejścia. Rezultaty sugerują, że przy odpowiednim doborze kombinacji augmentacji i segmentacji, metoda ta może przewyższać klasyczne podejścia takie jak STES, oferując lepszą elastyczność i adaptacyjność do różnych typów danych.

5. Podsumowanie

Celem projektu było zbadanie potencjału połączenia augmentacji danych z podejściem *curriculum learning* w kontekście modeli wizji komputerowej. Ze względu na ograniczenia czasowe, skupiliśmy się przede wszystkim na opracowaniu i przetestowaniu nowatorskiej metody augmentacji obrazów, opartej na nieregularnym podziale obrazu na fragmenty i stosowaniu lokalnych transformacji.

W ramach prac zaimplementowaliśmy dwie klasy augmentorów: bazujący na metodzie *STES* oraz autorską metodę *Augmentor*, która pozwalała na bardziej elastyczne i zróżnicowane przekształcenia poszczególnych segmentów obrazu. Nasze podejście charakteryzuje się niestandardowym, losowym podziałem obrazu na prostokątne regiony, co daje większą różnorodność generowanych danych treningowych.

Przeprowadzone eksperymenty obejmowały różne modele sieci neuronowych (*EfficientNet*, *MobileNetV2*) oraz popularne zbiory danych (*CIFAR-10*, *Fashion-MNIST*). Podczas testów zaobserwowaliśmy kilka istotnych wyzwań technicznych, takich jak błędy w przeskalowaniu obrazów i niestabilności w trakcie treningu modeli pretrenowanych, które udało się zidentyfikować i częściowo naprawić.

Ostateczne badania koncentrowały się na ocenie wpływu różnych technik augmentacji stosowanych lokalnie na fragmenty obrazów oraz na analizie konfiguracji podziałów obrazu. Chociaż nie udało się w pełni zweryfikować korzyści płynących z integracji augmentacji i *curriculum learning*, wyniki wskazują, że nieregularna augmentacja segmentów może znacząco zmieniać strukturę obrazu i potencjalnie wpłynąć na proces uczenia modeli.

Projekt dostarczył solidnej podstawy do dalszych badań i rozwoju metod augmentacji obrazów, a także zwrócił uwagę na ważne aspekty implementacyjne, które mają kluczowe znaczenie dla stabilności i skuteczności treningu modeli wizji komputerowej.

Bibliografia

[1] Albumentations. *Albumentations Documentation*. https://albumentations.ai/docs/.

[2] Juhwan Choi i Youngbin Kim. *Colorful Cutout: Curriculum Learning Guided Data Augmentation for Computer Vision*. 2024. URL: https://arxiv.org/pdf/2403.20012.

[3] Chengkai Hou, Jieyu Zhang i Tianyi Zhou. "When to Learn What: Model-Adaptive Data Augmentation Curriculum". W: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023. URL: https://openaccess.thecvf.com/content/ICCV2023/papers/Hou_When_to_Learn_What_Model-Adaptive_Data_Augmentation_Curriculum_ICCV_2023_paper.pdf.

[4] Shicai Huang, Jing Zhang i Xue Deng. "Self-transformation of encoded subregion image data augmentation for better classification". W: *Journal of Electronic Imaging* 33.2 (2024), s. 023046. DOI: 10.1117/1.JEI.33.2.023046. URL: https://doi.org/10.1117/1.JEI.33.2.023046.

[5] Sung-Jae Lee i in. "Enhancing deep learning classification performance of tongue lesions in imbalanced data: mosaic-based soft labeling with curriculum learning". W: *BMC Oral Health* 24.161 (2024). DOI: 10.1186/s12903-024-03898-3. URL: https://doi.org/10.1186/s12903-024-03898-3.

[6] OpenAI. *ChatGPT: GPT-4 model*. https://chat.openai.com/.

[7] PyTorch. *PyTorch Documentation*. https://pytorch.org/docs/stable/.